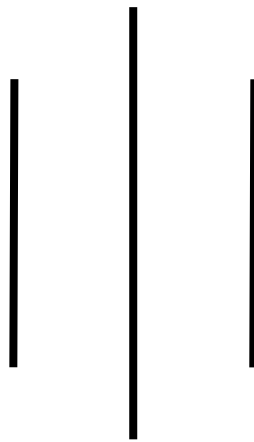


TRIBHUVAN UNIVERSITY

PATAN MULTIPLE CAMPUS

PATANDHOKA, LALITPUR



LAB REPORT

INFORMATION SECURITY (BIT 303)

SUBMITTED BY:

NAME: SURESH DAHAL

CLASS: BIT – III/I

ROLL: 23

DATE: 2082-.....-.....

SUBMITTED TO:

DADHI GHIMIRE

CHECKED BY

1. Implement Caesar cipher to encrypt and decrypt message using a key k.

THEORY:

Caesar Cipher is a simple encryption technique where each letter in the plaintext is shifted by a fixed number of positions in the alphabet using key k.

Formula:

Encryption:

$$C = (P + k) \bmod 26$$

Decryption:

$$P = (C - k) \bmod 26$$

Where:

- P= Plaintext letter position
- C= Ciphertext letter position
- k= Key

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void encrypt(char text[], int key)
{
    for (int i = 0; text[i] != '\0'; i++)
    {
        char ch = text[i];

        if (isalpha(ch))
        {
            char base = islower(ch) ? 'a' : 'A';
            text[i] = (ch - base + key) % 26 + base;
        }
    }
}

void decrypt(char text[], int key)
```

```

{
    for (int i = 0; text[i] != '\0'; i++)
    {
        char ch = text[i];

        if (isalpha(ch))
        {
            char base = islower(ch) ? 'a' : 'A';
            text[i] = (ch - base - key + 26) % 26 + base;
        }
    }
}

int main()
{
    char text[100];
    int key, choice;

    printf("Enter message: ");
    gets(text);

    printf("Enter key: ");
    scanf("%d", &key);

    printf("1. Encrypt\n2. Decrypt\nEnter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        encrypt(text, key);
        printf("Encrypted message: %s", text);
    }
    else if (choice == 2)
    {
        decrypt(text, key);
        printf("Decrypted message: %s", text);
    }
    else
    {
        printf("Invalid choice");
    }

    return 0;
}

```

OUTPUT:

```
PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter message: suresh
Enter key: 3
1. Encrypt
2. Decrypt
Enter choice: 1
Encrypted message: vxuhvk

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter message: vxuhvk
Enter key: 3
1. Encrypt
2. Decrypt
Enter choice: 2
Decrypted message: suresh
```

2. IMPLEMENT HILL CIPHER TO ENCRYPT AND DECRYPT THE WORD “GOOD” USING KEY MATRIX $\begin{bmatrix} 7 & 3 \\ 7 & 4 \end{bmatrix}$.

THEORY:

Hill cipher is a type of block cipher that uses matrix multiplication to encrypt a group of letters simultaneously. In this cipher:

- Each letter is treated as a number
- The plaintext is divided into vectors of size n
- Encryption is done by multiplying the plaintext vector by a square matrix of key and taking the result modulo 26.
- Decryption requires finding the inverse of the key matrix.

Formula:

- **Encryption:** $C = (K \times P) \pmod{26}$
- **Decryption:** $P = (K^{-1} \times C) \pmod{26}$

Where K is the key matrix, P is the plaintext vector, and C is the ciphertext vector.

PROGRAM:

```
#include <stdio.h>

int main()
{
    int key[2][2] = {{7, 3}, {7, 4}};
    char word[] = "GOOD";
```

```

int p[2], c[2];
int i, j;
int det = (key[0][0] * key[1][1]) - (key[0][1] * key[1][0]);
det = det % 26;

int det_inv = 0;
for (i = 0; i < 26; i++)
{
    if ((det * i) % 26 == 1)
    {
        det_inv = i;
        break;
    }
}

int invKey[2][2];
invKey[0][0] = (key[1][1] * det_inv) % 26;
invKey[0][1] = ((-key[0][1] + 26) * det_inv) % 26;
invKey[1][0] = ((-key[1][0] + 26) * det_inv) % 26;
invKey[1][1] = (key[0][0] * det_inv) % 26;

printf("--- HILL CIPHER CALCULATION ---\n");
printf("Step 1: Key Matrix is [[%d, %d], [%d, %d]]\n", key[0][0],
key[0][1], key[1][0], key[1][1]);
printf("Step 2: Determinant = %d, Modular Inverse = %d\n\n", det,
det_inv);

printf("--- ENCRYPTION ---\n");
for (i = 0; i < 4; i += 2)
{
    p[0] = word[i] - 'A';
    p[1] = word[i + 1] - 'A';

    for (j = 0; j < 2; j++)
    {
        int sum = (key[j][0] * p[0] + key[j][1] * p[1]);
        c[j] = sum % 26;
        printf("Calculation for %c: (%d*%d + %d*%d) mod 26 = %d (%c)\n",
            word[i + j], key[j][0], p[0], key[j][1], p[1], c[j], c[j] +
'A');
    }
    word[i] = c[0] + 'A';
    word[i + 1] = c[1] + 'A';
    printf("Resulting Block: %c%c\n\n", word[i], word[i + 1]);
}

printf("--- DECRYPTION ---\n");
for (i = 0; i < 4; i += 2)

```

```

{
    c[0] = word[i] - 'A';
    c[1] = word[i + 1] - 'A';

    for (j = 0; j < 2; j++)
    {
        int sum = (invKey[j][0] * c[0] + invKey[j][1] * c[1]);
        int res = sum % 26;
        printf("Calculation for %c: (%d*%d + %d*%d) mod 26 = %d (%c)\n",
            word[i + j], invKey[j][0], c[0], invKey[j][1], c[1], res,
res + 'A');
    }
    printf("\n");
}

return 0;
}

```

OUTPUT:

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe

--- HILL CIPHER CALCULATION ---

Step 1: Key Matrix is [[7, 3], [7, 4]]

Step 2: Determinant = 7, Modular Inverse = 15

--- ENCRYPTION ---

Calculation for G: $(7*6 + 3*14) \bmod 26 = 6$ (G)

Calculation for O: $(7*6 + 4*14) \bmod 26 = 20$ (U)

Resulting Block: GU

Calculation for O: $(7*14 + 3*3) \bmod 26 = 3$ (D)

Calculation for D: $(7*14 + 4*3) \bmod 26 = 6$ (G)

Resulting Block: DG

--- DECRYPTION ---

Calculation for G: $(8*6 + 7*20) \bmod 26 = 6$ (G)

Calculation for U: $(25*6 + 1*20) \bmod 26 = 14$ (O)

Calculation for D: $(8*3 + 7*6) \bmod 26 = 14$ (O)

Calculation for G: $(25*3 + 1*6) \bmod 26 = 3$ (D)

3. Write a program to encrypt and decrypt the message “nocheating” with key “exam” using vigenère cipher.

THEORY:

The Vigenère Cipher is a type of cipher where each letter of the plaintext is shifted by a value determined by a corresponding letter in the keyword. Unlike a standard Caesar cipher that uses a single shift for the entire message, Vigenère uses a different shift for each character. If the keyword is shorter than the message, it is repeated cyclically.

Formula

- Encryption: $C_i = (P_i + K_i) \pmod{26}$
- Decryption: $P_i = (C_i - K_i + 26) \pmod{26}$

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char message[] = "NOCHEATING";
    char key[] = "EXAM";
    int msgLen = strlen(message);
    int keyLen = strlen(key);
    char cipher[20], decrypted[20];
    int i;

    for (i = 0; i < msgLen; i++)
    {
        int p = message[i] - 'A';
        int k = key[i % keyLen] - 'A';

        cipher[i] = ((p + k) % 26) + 'A';
    }
    cipher[msgLen] = '\0';

    for (i = 0; i < msgLen; i++)
    {
        int c = cipher[i] - 'A';
        int k = key[i % keyLen] - 'A';

        decrypted[i] = ((c - k + 26) % 26) + 'A';
    }
    decrypted[msgLen] = '\0';

    printf("Plaintext: %s\n", message);
```

```

printf("Keyword:    %s\n", key);
printf("Ciphertext: %s\n", cipher);
printf("Decrypted:  %s\n", decrypted);

return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Plaintext:  NOCHEATING
Keyword:    EXAM
Ciphertext: RLCTIXTURD
Decrypted:  NOCHEATING

```

4. Implement rail-fence cipher to encrypt and decrypt a message “your own message” with fence size k.

THEORY:

The Rail-Fence cipher does not change the letters themselves but rearranges their order. The message is written diagonally in a zigzag pattern on imaginary rails and then read off row by row.

- **Fence Size (k):** This determines the number of horizontal rows or rails used to write the message.
- **Mechanism:** The characters are placed in a V-shape pattern. Once the grid is filled, the ciphertext is formed by reading the characters from the first rail, then the second, and so on.

Formula/Logic

To find the position of a character in a rail at index i with key k :

- Determine the direction (downward or upward).
- The direction changes when we hit the top rail (0) or the bottom rail ($k-1$).

PROGRAM:

```

#include <stdio.h>
#include <string.h>

int main()
{
    char msg[] = "SURESH";

```



```

int k = 3;
int msgLen = strlen(msg);
char rail[k][msgLen];
int i, j, row, check;

for (i = 0; i < k; i++)
    for (j = 0; j < msgLen; j++)
        rail[i][j] = '\n';

row = 0;
check = 0;
for (i = 0; i < msgLen; i++)
{
    rail[row][i] = msg[i];

    if (row == 0)
        check = 0;
    else if (row == k - 1)
        check = 1;

    if (check == 0)
        row++;
    else
        row--;
}

printf("Ciphertext: ");
for (i = 0; i < k; i++)
{
    for (j = 0; j < msgLen; j++)
    {
        if (rail[i][j] != '\n')
            printf("%c", rail[i][j]);
    }
}

printf("\nDecrypted: ");
row = 0;
check = 0;
for (i = 0; i < msgLen; i++)
{
    printf("%c", rail[row][i]);

    if (row == 0)
        check = 0;
    else if (row == k - 1)
        check = 1;
}

```

```

        if (check == 0)
            row++;
        else
            row--;
    }
    printf("\n");

    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Ciphertext: SSUEHR
Decrypted: SURESH

```

5. Implement playfair cipher to encrypt and decrypt a message “this is kathmandu” using key “nepal”.

THEORY:

The Playfair Cipher is a digraph substitution cipher. Instead of shifting single letters, it encrypts letters in group of two. This makes it more secure than simple substitution ciphers.

- The 5x5 Matrix We create a grid using the key "NEPAL" first, then fill the rest with the remaining alphabet. Since there are 26 letters and only 25 slots, I and J share a single cell.
- **The Rules of Encryption**
 - Same Row: Replace each letter with the one to its immediate right.
 - Same Column: Replace each letter with the one immediately below it.
 - Rectangle Rule: if letters are in different rows/columns, they form a rectangle. Swap them with the letters in the opposite corners of that rectangle.

PROGRAM:

```

#include <stdio.h>
#include <string.h>

int main() {
    char matrix[5][5] = {
        {'N', 'E', 'P', 'A', 'L'},
        {'B', 'C', 'D', 'F', 'G'},
        {'H', 'I', 'K', 'M', 'O'},
        {'Q', 'R', 'S', 'T', 'U'},
        {'V', 'W', 'X', 'Y', 'Z'}};
}

```

```

char msg[] = "THISISKATHMANDU";
int len = strlen(msg);
int r1, c1, r2, c2;

printf("Key: NEPAL | Message: %s\n", msg);
printf("Ciphertext: ");

for (int i = 0; i < len; i += 2)
{
    char a = msg[i];
    char b = (i + 1 < len) ? msg[i + 1] : 'X';

    for (int r = 0; r < 5; r++)
    {
        for (int c = 0; c < 5; c++)
        {
            if (matrix[r][c] == a)
            {
                r1 = r;
                c1 = c;
            }
            if (matrix[r][c] == b)
            {
                r2 = r;
                c2 = c;
            }
        }
    }

    if (r1 == r2)
        printf("%c%c", matrix[r1][(c1 + 1) % 5], matrix[r2][(c2 + 1) %
5]);
    else if (c1 == c2)
        printf("%c%c", matrix[(r1 + 1) % 5][c1], matrix[(r2 + 1) %
5][c2]);
    else
        printf("%c%c", matrix[r1][c2], matrix[r2][c1]);
    }
    printf("\n");
    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Key: NEPAL | Message: THISISKATHMANDU
Ciphertext: QMKRKRMPQMTFPBSZ

```

6. Implement one-time pad algorithm to encrypt and decrypt a message.

THEORY:

The One-Time Pad is a symmetric key encryption technique that is considered theoretically unbreakable when used correctly. It encrypts plain text by combining it with a random secret key of the same length.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int main() {
    char plaintext[] = "SURESH";
    char key[] = "XMCKLZ";
    int len = strlen(plaintext);
    char ciphertext[len + 1], decrypted[len + 1];
    int i;

    printf("Plaintext: %s\n", plaintext);
    printf("Key:          %s\n\n", key);

    for (i = 0; i < len; i++)
    {
        int p = plaintext[i] - 'A';
        int k = key[i] - 'A';

        ciphertext[i] = ((p + k) % 26) + 'A';
    }
    ciphertext[len] = '\0';
    printf("Ciphertext: %s\n", ciphertext);

    for (i = 0; i < len; i++)
    {
        int c = ciphertext[i] - 'A';
        int k = key[i] - 'A';

        decrypted[i] = ((c - k + 26) % 26) + 'A';
    }

    decrypted[len] = '\0';
    printf("Decrypted:  %s\n", decrypted);

    return 0;
}
```

OUTPUT:

```
PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Plaintext: SURESH
Key:      XMCKLZ

Ciphertext: PGTODG
Decrypted:  SURESH
```

7. Implement extended Euclidean algorithm to find multiplicative inverse of an integer mod n.

THEORY:

The Extended Euclidean Algorithm is an extension of the standard Euclidean Algorithm. While the standard version only finds the Greatest Common Divisor (GCD) of two integers a and b, the extended version also finds the coefficients x and y such that:

$$ax + by = \text{GCD}(a, b)$$

In cryptography, this is primarily used to find the multiplicative inverse. An integer x is the multiplicative inverse of a mod n if:

$$ax \equiv 1 \pmod{n}$$

PROGRAM:

```
#include <stdio.h>

int extendedGCD(int a, int n) {
    int m0 = n, t, q;
    int x0 = 0, x1 = 1;
    if (n == 1) return 0;

    while (a > 1) {
        q = a / n;
        t = n;

        n = a % n;
        a = t;
        t = x0;

        x0 = x1 - q * x0;
        x1 = t;
    }

    if (x1 < 0) x1 += m0;
    return x1;
}
```

```

int main() {
    int a = 11, n = 26;

    printf("Finding Multiplicative Inverse of %d mod %d...\n", a, n);

    int inverse = extendedGCD(a, n);

    printf("The Multiplicative Inverse is: %d\n", inverse);
    printf("Check: (%d * %d) %% %d = %d\n", a, inverse, n, (a * inverse) % n);

    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Finding Multiplicative Inverse of 11 mod 26...
The Multiplicative Inverse is: 19
Check: (11 * 19) % 26 = 1

```

8. Write a program to find all the primitive root of prime number p.

THEORY:

A primitive root modulo p is a number g such that its powers generate all numbers from 1 to $p - 1$ when taken modulo p.

That means:

$$g^1, g^2, g^3, \dots, g^{p-1} \bmod p$$

gives all values from 1 to $p - 1$ without repeating.

Algorithm:

- a) Input prime number p
- b) Compute $n = p - 1$
- c) Find prime factors of n
- d) For each number $g = 2$ to $p - 1$:
 - Calculate $g^{n/q} \bmod p$ for each factor q
 - If result is 1 for any, skip it
 - Otherwise, it is a primitive root
- e) Print all such values

PROGRAM:

```
#include <stdio.h>
#include <math.h>

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            res = (res * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }
    return res;
}

int isPrimitiveRoot(int g, int P) {
    int values[P];
    for (int i = 0; i < P; i++)
        values[i] = 0;

    long long current = 1;
    for (int i = 1; i < P; i++) {
        current = (current * g) % P;
        if (values[current] == 1)
            return 0;
        values[current] = 1;
    }
    return 1;
}

int main() {
    int P = 7;
    printf("Primitive roots of %d are: ", P);

    for (int g = 2; g < P; g++)
    {
        if (isPrimitiveRoot(g, P))
        {
            printf("%d ", g);
        }
    }
    printf("\n");

    return 0;
}
```

OUTPUT:

```
PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Primitive roots of 7 are: 3 5
```

9. Write a program to find totient number of given integer n.

THEORY:

Euler's Totient Function, denoted as $\phi(n)$, is a fundamental function in number theory and cryptography. It counts the number of positive integers up to n that are relatively prime to n .

Working Principle

- **Relatively Prime:** Two numbers a and n are relatively prime if their Greatest Common Divisor is 1.
- **Definition:** For a given integer n , $\phi(n)$ is the count of numbers i such that $1 \leq i < n$ and $\gcd(i, n) = 1$.
- **Prime Numbers:** If n is a prime number P , then all numbers from 1 to $P-1$ are relatively prime to it, so $\phi(P) = P - 1$.

PROGRAM:

```
#include <stdio.h>

int gcd(int a, int b)
{
    if (a == 0)
        return b;
    return gcd(b % a, a);
}

int phi(int n)
{
    int result = 1;
    for (int i = 2; i < n; i++)
    {
        if (gcd(i, n) == 1)
        {
            result++;
        }
    }
}
```



```

    }
    return result;
}

int main()
{
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);

    printf("Euler's Totient Number phi(%d) = %d\n", n, phi(n));

    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter the value of n: 9
Euler's Totient Number phi(9) = 6

```

10. Write a program to implement Fermat's theorem $a^{p-1} \equiv 1 \pmod{p}$. (Where a and p are relatively prime)

THEORY:

Fermat's Little Theorem is a fundamental theorem in number theory, which provides a property of prime numbers. It states that if p is a prime number and a is any integer such that a is not divisible by p (meaning a and p are relatively prime), then:

$$a^{p-1} \equiv 1 \pmod{p}$$

PROGRAM:

```

#include <stdio.h>

long long power(long long base, long long exp, long long mod)
{
    long long res = 1;
    base = base % mod;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            res = (res * base) % mod;
        base = (base * base) % mod;
    }
}

```

```

        exp = exp / 2;
    }
    return res;
}

int gcd(int a, int b)
{
    if (a == 0)
        return b;
    return gcd(b % a, a);
}

int main()
{
    int a, p;

    printf("Enter an integer (a): ");
    scanf("%d", &a);
    printf("Enter a prime number (p): ");
    scanf("%d", &p);

    if (gcd(a, p) != 1)
    {
        printf("Error: a and p must be relatively prime (gcd=1).\n");
    }
    else
    {
        long long result = power(a, p - 1, p);

        printf("\nVerification of Fermat's Little Theorem:\n");
        printf("%d^(%d-1) mod %d = %lld\n", a, p, p, result);

        if (result == 1)
        {
            printf("Result is 1. Theorem holds true.\n");
        }
        else
        {
            printf("Theorem failed (p might not be prime).\n");
        }
    }

    return 0;
}

```

OUTPUT:

```
PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter an integer (a): 8
Enter a prime number (p): 3

Verification of Fermat's Little Theorem:
8^(3-1) mod 3 = 1
Result is 1. Theorem holds true.
```

11. Implement Miller Rabin algorithm to determine whether a given odd integer is composite or may be prime (inconclusive).

THEORY:

The Miller-Rabin algorithm is a probabilistic primality test used to determine if a given odd integer n is composite or probably prime. Unlike deterministic tests, it uses random bases to check for primality, making it much faster for very large numbers.

Working Principle

- **Factorization:** We write $(n - 1)$ as $(2^s * d)$, where d is an odd number.
- **Witness Selection:** We pick a random integer 'a' between 2 and $n - 2$.
- **The Test:** We compute $x = (a^d) \bmod n$.
 - If $x = 1$ or $x = n - 1$, the number n is probably prime for this witness.
 - If not, we square x repeatedly up to $s-1$ times. If any resulting x equals $n - 1$, the number is probably prime.
- **Inconclusive vs. Composite:** If none of these conditions are met, the number is definitely composite. If they are met, we repeat the test with different values of 'a' to increase confidence.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

long long power(long long base, long long exp, long long mod)
{
    long long res = 1;
    base %= mod;
```

```

while (exp > 0)
{
    if (exp % 2 == 1)
        res = (res * base) % mod;
    base = (base * base) % mod;
    exp /= 2;
}
return res;
}

int millerRabin(long long n, int k)
{
    if (n <= 1 || n == 4)
        return 0;
    if (n <= 3)
        return 1;

    long long d = n - 1;
    int s = 0;
    while (d % 2 == 0)
    {
        d /= 2;
        s++;
    }

    for (int i = 0; i < k; i++)
    {
        long long a = 2 + rand() % (n - 4);
        long long x = power(a, d, n);

        if (x == 1 || x == n - 1)
            continue;

        int composite = 1;
        for (int r = 1; r < s; r++)
        {
            x = (x * x) % n;
            if (x == n - 1)
            {
                composite = 0;
                break;
            }
        }
        if (composite)
            return 0;
    }
    return 1;
}

```

```

int main()
{
    long long n;
    int k = 5;

    printf("Enter an odd integer: ");
    scanf("%lld", &n);

    if (n % 2 == 0 && n != 2)
    {
        printf("%lld is Composite.\n", n);
    }
    else if (millerRabin(n, k)) {
        printf("%lld is Probably Prime (Inconclusive).\n", n);
    }
    else {
        printf("%lld is Composite.\n", n);
    }

    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter an odd integer: 15
15 is Composite.

```

12.WAP to establish a private key using Diffie Hellman key Exchange.

THEORY:

The Diffie-Hellman key exchange is a method for two parties to securely establish a shared private key over a public channel. It is not an encryption algorithm itself, but a way to agree on a secret key that can later be used for symmetric encryption like AES.

Working Principle:

Two users (say **A** and **B**) agree on:

- A prime number p
- A primitive root g

These values are public.

Steps:

Step 1: Private Keys

- A selects private key a
- B selects private key b

Step 2: Public Keys

- A computes:

$$A = g^a \text{mod } p$$

- B computes:

$$B = g^b \text{mod } p$$

They exchange these values.

Step 3: Shared Secret Key

- A computes:

$$K = B^a \text{mod } p$$

- B computes:

$$K = A^b \text{mod } p$$

Both get the same key:

$$K = g^{ab} \text{mod } p$$

PROGRAM:

```
#include <stdio.h>

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            res = (res * base) % mod;
    }
}
```

```

        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

int main() {
    long long q, g, Xa, Xb, Ya, Yb, Ka, Kb;

    q = 23;
    g = 5;
    printf("Publicly Shared Prime (q): %lld\n", q);
    printf("Publicly Shared Root (g): %lld\n\n", g);

    Xa = 6;
    Xb = 15;
    printf("User A Private Key (Xa): %lld\n", Xa);
    printf("User B Private Key (Xb): %lld\n\n", Xb);

    Ya = power(g, Xa, q);
    Yb = power(g, Xb, q);
    printf("User A Public Value (Ya): %lld\n", Ya);
    printf("User B Public Value (Yb): %lld\n\n", Yb);

    Ka = power(Yb, Xa, q);
    Kb = power(Ya, Xb, q);

    printf("Secret Key calculated by User A: %lld\n", Ka);
    printf("Secret Key calculated by User B: %lld\n", Kb);

    if (Ka == Kb) {
        printf("\nSuccess: Shared Private Key established!\n");
    }
    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Publicly Shared Prime (q): 23
Publicly Shared Root (g): 5

User A Private Key (Xa): 6
User B Private Key (Xb): 15

User A Public Value (Ya): 8
User B Public Value (Yb): 19

Secret Key calculated by User A: 2
Secret Key calculated by User B: 2

Success: Shared Private Key established!

```

13. Write a program to implement RSA algorithm to encrypt a message.

THEORY:

RSA (Rivest-Shamir-Adleman) is an asymmetric cryptographic algorithm. It uses a public key for encryption and a private key for decryption. The security of RSA is built on the mathematical difficulty of factoring the product of two large prime numbers.

Working Principle

➤ Key Generation:

1. Select two distinct prime numbers, p and q .
2. Compute $n = p * q$. This n is the modulus for both keys.
3. Compute Euler's Totient function: $\phi(n) = (p - 1) * (q - 1)$.
4. Choose an integer e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. (Commonly $e = 65537$).
5. Compute d such that $(d * e) \% \phi(n) = 1$. (This uses the Extended Euclidean Algorithm).

➤ **Encryption:** The ciphertext C is calculated as: $C = (M^e) \bmod n$.

➤ **Decryption:** The plaintext message M is recovered as: $M = (C^d) \bmod n$.

PROGRAM:

```
#include <stdio.h>
#include <math.h>

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
```



```

        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    long long p = 3, q = 11;
    long long n = p * q;
    long long phi = (p - 1) * (q - 1);

    long long e = 7;
    while (e < phi) {
        if (gcd(e, phi) == 1)
            break;
        else
            e++;
    }

    long long d = 0;
    for (long long i = 1; i < phi; i++) {
        if ((e * i) % phi == 1) {
            d = i;
            break;
        }
    }

    long long msg = 65;
    printf("Original Message: %lld\n", msg);
    printf("Public Key (e, n): (%lld, %lld)\n", e, n);
    printf("Private Key (d, n): (%lld, %lld)\n\n", d, n);

    long long c = power(msg, e, n);
    printf("Encrypted Message (Ciphertext): %lld\n", c);

    long long m = power(c, d, n);
    printf("Decrypted Message (Plaintext): %lld\n", m);

    return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Original Message: 65
Public Key (e, n): (7, 33)
Private Key (d, n): (3, 33)

Encrypted Message (Ciphertext): 32
Decrypted Message (Plaintext): 32

```

14. Implement RSA based Digital signature to sign and verify a digital signature.

THEORY:

A Digital Signature is a mathematical technique used to validate the authenticity and integrity of a message, software, or digital document. While RSA encryption uses the public key to encrypt and the private key to decrypt, a Digital Signature reverses this process:

- **Signing:** The sender uses their Private Key to encrypt a message digest. This locks the identity of the sender to the message.
- **Verification:** The receiver uses the sender's Public Key to decrypt the signature. If the result matches the original message, the signature is valid.

PROGRAM:

```
#include <stdio.h>

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1)
            res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

int main() {
    long long p = 3, q = 11;
    long long n = p * q;
    long long phi = (p - 1) * (q - 1);
    long long e = 7;
    long long d = 3;

    long long M = 14;

    printf("Original Message (M): %lld\n", M);
    printf("Public Key (e, n): (%lld, %lld)\n", e, n);
    printf("Private Key (d, n): (%lld, %lld)\n\n", d, n);
```

```

long long S = power(M, d, n);
printf("Digital Signature (S) created: %lld\n", S);

long long M_prime = power(S, e, n);
printf("Decrypted value from signature: %lld\n\n", M_prime);

if (M_prime == M) {
    printf("Verification SUCCESS: The signature is valid and
authentic.\n");
}
else {
    printf("Verification FAILED: The signature or message has been
tampered with.\n");
}

return 0;
}

```

OUTPUT:

```

PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> .\a.exe
Original Message (M): 14
Public Key (e, n): (7, 33)
Private Key (d, n): (3, 33)

Digital Signature (S) created: 5
Decrypted value from signature: 14

Verification SUCCESS: The signature is valid and authentic.

```

15. Implement DES algorithm in python.

THEORY:

DES (Data Encryption Standard) is a symmetric-key block cipher that encrypts data in 64-bit blocks using a 56-bit key. It follows a Feistel Network structure, meaning it splits the block into two halves and applies a series of transformations over 16 rounds.

Working Principle

- **Initial Permutation (IP):** The 64-bit input block is rearranged according to a fixed table.
- **Key Schedule:** The 64-bit input key is reduced to 56 bits and then shifted and permuted to create sixteen 48-bit "round keys."

- **Feistel Rounds:** In each of the 16 rounds, the right half of the data is expanded, XORed with the round key, passed through **S-Boxes** (Substitution), and permuted.
- **Final Permutation (FP):** The inverse of the IP is applied to produce the ciphertext.

PROGRAM:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

def encrypt(msg, key):
    cipher = DES.new(key, DES.MODE_ECB)
    padded_text = pad(msg.encode(), 8)
    encrypted_text = cipher.encrypt(padded_text)
    return encrypted_text

def decrypt(cipher_text, key):
    cipher = DES.new(key, DES.MODE_ECB)
    decrypted_text = unpad(cipher.decrypt(cipher_text), 8)
    return decrypted_text.decode()

if __name__ == "__main__":
    key = b'8bytekey'
    message = input("Enter message: ")

    encrypted = encrypt(message, key)
    print("Encrypted (in bytes):", encrypted.hex())

    decrypted = decrypt(encrypted, key)
    print("Decrypted message:", decrypted)
```

OUTPUT:

```
VICTUS@Aconitin MINGW64 /c/TURBOC3/Projects/Suresh/IS-LAB-Suresh
$ python 15.\des.py
bash: sed: command not found
Enter message: suresh
Encrypted (in bytes): 856236b94cbec970
Decrypted message: suresh
```

16. Implement AES algorithm in python.

THEORY:

The Advanced Encryption Standard (AES) is a symmetric key encryption algorithm used worldwide for securing data. It is stronger and more secure than DES. Unlike DES, which uses a Feistel network, AES uses a Substitution-Permutation Network.

Key Features

- Block size: 128 bits
- Key sizes:
 - 128-bit
 - 192-bit
 - 256-bit

PROGRAM:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

def encrypt(message, key):
    cipher = AES.new(key, AES.MODE_ECB)
    padded_msg = pad(message.encode(), 16)
    encrypted = cipher.encrypt(padded_msg)
    return encrypted

def decrypt(cipher_text, key):
    cipher = AES.new(key, AES.MODE_ECB)
    decrypted = unpad(cipher.decrypt(cipher_text), 16)
    return decrypted.decode()

if __name__ == "__main__":
    key = b'1234567890123456'
    message = input("Enter message: ")

    encrypted = encrypt(message, key)
    print("Encrypted (bytes):", encrypted)

    decrypted = decrypt(encrypted, key)
    print("Decrypted message:", decrypted)
```

OUTPUT:

```
(.venv) PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> python '.\16. aes.py'
Enter message: suresh
Encrypted (bytes): 35f0860408d5ecabf0f53be34ed2e644
Decrypted message: suresh
```

17. Implement MD5 algorithm in python.

THEORY:

MD5 is a cryptographic hash function that produces a 128-bit (16-byte) hash value, usually represented as a 32-digit hexadecimal number. It was designed to be a cryptographic hash function for verifying data integrity.

How it works:

- **Padding:** The message is padded so its length is congruent to 448 mod 512.
- **Append Length:** A 64-bit representation of the original length is added.
- **Initialize Variables:** Four 32-bit registers (A, B, C, D) are initialized with specific constants.
- **Process Blocks:** The message is processed in 512-bit blocks. Each block goes through 4 rounds of complex logical operations (F, G, H, I function).
- **Output:** The final values in the registers are concatenated to form the hash.

PROGRAM:

```
import hashlib

def generate_md5(message):
    result = hashlib.md5(message.encode())
    return result.hexdigest()

if __name__ == "__main__":
    message = input("Enter message: ")

    md5_hash = generate_md5(message)
    print("MD5 Hash:", md5_hash)
```

OUTPUT:

```
(.venv) PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh>
Enter message: suresh
MD5 Hash: 0487cc982f7db39c51695026e4bdc692
```

18. Implement SHA 1 algorithm in python.

THEORY:

SHA-1 is a cryptographic hash function that converts any input data into a fixed-size 160-bit hash value. It is used for data integrity verification.

Working Principle

- Pad the message to make its length a multiple of 512 bits
- Split the message into 512-bit blocks
- Process each block using SHA-1 compression function
- Combine results to produce a 160-bit hash

PROGRAM:

```
import hashlib

def generate_sha1(message):
    result = hashlib.sha1(message.encode())
    return result.hexdigest()

if __name__ == "__main__":
    message = input("Enter message: ")
    sha1_hash = generate_sha1(message)
    print("SHA-1 Hash:", sha1_hash)
```

OUTPUT:

```
(.venv) PS C:\TURBOC3\Projects\Suresh\IS-LAB-Suresh> python
Enter message: suresh
SHA-1 Hash: 673a3ca2ac85faf699848fd436fdf6648ff02f2f
```